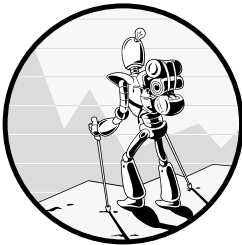


4

NON-NUMERIC VALUES



So far, you’ve been working almost exclusively with numeric values. But statistical programming also requires non-numeric values. In this chapter, we’ll consider four such data types: logicals, characters, factors, and date-times. These play an important role in effective use of R, especially as we get into more complex programming in Part II.

4.1 Logical Values

Logical values (which we also call *logicals*) are based on a simple premise: A logical-valued object can be either TRUE or FALSE. We interpret these as yes/no, one/zero, satisfied/not satisfied, and so on. This concept appears across all programming languages. Often, logical values signal whether a condition has been satisfied or whether a parameter should be switched on.

You encountered logical values briefly when you used the `sort` function in Section 2.3.2 and the `matrix` function in Section 3.1. When using `sort`, setting `decreasing=TRUE` returns a vector ordered from largest to smallest, and `decreasing=FALSE` sorts the vector the other way around. Similarly, when

constructing a matrix, `byrow=TRUE` fills the matrix entries row-wise; otherwise, the matrix fills column-wise. Let's take a more detailed look at ways to use logicals.

4.1.1 *TRUE or FALSE?*

We write logical values in R as `TRUE` and `FALSE`, though we can abbreviate them as `T` or `F`. The abbreviated version has no effect on the execution of the code; for example, using `decreasing=T` is equivalent to `decreasing=TRUE` in the `sort` function. (Note, however, that you shouldn't name other objects `T` or `F` in your workspace if you plan to use this convenience—see Section 9.1.3.)

Assigning logical values to an object is the same as assigning numeric values:

```
R> foo <- TRUE
R> foo
[1] TRUE
R> bar <- F
R> bar
[1] FALSE
```

This gives you one object with the value `TRUE` and one with the value `FALSE`. Similarly, you can fill vectors with logical values:

```
R> baz <- c(T,F,F,F,T,F,T,T,T,F,T,F)
R> baz
[1] TRUE FALSE FALSE FALSE TRUE FALSE TRUE TRUE TRUE FALSE TRUE FALSE
R> length(x=baz)
[1] 12
```

You can create matrices (and other higher-dimensional arrays) with these values, too. Using `foo` and `baz`, you might construct something like this:

```
R> qux <- matrix(data=baz,nrow=3,ncol=4,byrow=foo)
R> qux
      [,1] [,2] [,3] [,4]
[1,] TRUE FALSE FALSE FALSE
[2,] TRUE FALSE TRUE TRUE
[3,] TRUE FALSE TRUE FALSE
```

As expected given the use of `byrow` and the stored value of `foo`, the resulting structure has been filled in a row-wise fashion.

4.1.2 *Relational Operators*

We commonly use logicals to check relationships between values. For example, to test whether some number a is greater than a predefined threshold b , use the standard relational operators in Table 4-1, which produce logical values as results.

Table 4-1: Relational Operators

Operator	Interpretation
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Typically, we use these operators on numeric values (though you'll look at some other possibilities in Section 4.2.1). Here's an example:

```
R> 1==2
[1] FALSE
R> 1>2
[1] FALSE
R> (2-1)<=2
[1] TRUE
R> 1!=(2+3)
[1] TRUE
```

The results should be unsurprising: 1 being equal to 2 is FALSE and 1 being greater than 2 is also FALSE, while it's TRUE that the result of 2-1 is less than or equal to 2, and that 1 *isn't* equal to 5 (2+3). These operations are much more useful when used on numbers that are variable in some way, as you'll see shortly.

You're already familiar with R's element-wise behavior when working with vectors. The same rules apply when using relational operators. To illustrate this, first create two vectors and double-check that they're of equal length:

```
R> foo <- c(3,2,1,4,1,2,1,-1,0,3)
R> bar <- c(4,1,2,1,1,0,0,3,0,4)
R> length(x=foo)==length(x=bar)
[1] TRUE
```

Now consider the following four evaluations:

```
R> foo==bar
[1] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE TRUE FALSE
R> foo<bar
[1] TRUE FALSE TRUE FALSE FALSE FALSE FALSE TRUE FALSE TRUE
R> foo<=bar
[1] TRUE FALSE TRUE FALSE TRUE FALSE FALSE TRUE TRUE TRUE
R> foo<=(bar+10)
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

The first line checks whether the entries in `foo` are equal to the corresponding entries in `bar`, which is true only for the fifth and ninth entries. As the returned vector contains a logical result for each pair of elements, it's the same length as the vectors we're comparing. The second line compares `foo` and `bar` in the same way, this time checking whether the entries in `foo` are less than the entries in `bar`. Contrast this result with the third comparison, which asks whether entries are less than *or equal to* one another. Finally, the fourth line checks whether `foo`'s members are less than or equal to `bar`, when the elements of `bar` are increased by 10. Naturally, all of the results are `TRUE`.

Vector recycling also applies to logicals. Let's use `foo` from earlier, along with a shorter vector, `baz`:

```
R> baz <- foo[c(10,3)]
R> baz
[1] 3 1
```

You create `baz` as a vector of length 2 consisting of the tenth and third elements of `foo`. Now consider the following:

```
R> foo>baz
[1] FALSE TRUE FALSE TRUE FALSE TRUE FALSE FALSE TRUE
```

Here, the 2 elements of `baz` are recycled and checked against the 10 elements of `foo`. Elements 1 and 2 of `foo` are checked against 1 and 2 of `baz`, elements 3 and 4 of `foo` are checked against 1 and 2 of `baz`, and so on. You can also check all the values of a vector against a single value. Here's an example:

```
R> foo<3
[1] FALSE TRUE TRUE FALSE TRUE TRUE TRUE TRUE FALSE
```

This kind of operation will become familiar when handling and subsetting data frames in Chapter 5.

For now, let's rewrite the contents of `foo` and `bar` as 5×2 column-filled matrices:

```
R> foo.mat <- matrix(foo,nrow=5,ncol=2)
R> foo.mat
      [,1] [,2]
[1,]    3    2
[2,]    2    1
[3,]    1   -1
[4,]    4    0
[5,]    1    3
R> bar.mat <- matrix(bar,nrow=5,ncol=2)
R> bar.mat
      [,1] [,2]
[1,]    4    0
[2,]    1    0
[3,]    2    3
```

```
[4,] 1 0
[5,] 1 4
```

The same element-wise behavior applies here; if you compare the matrices, you get a matrix of the same size filled with logicals:

```
R> foo.mat<=bar.mat
      [,1] [,2]
[1,] TRUE FALSE
[2,] FALSE FALSE
[3,] TRUE TRUE
[4,] FALSE TRUE
[5,] TRUE TRUE
R> foo.mat<3
      [,1] [,2]
[1,] FALSE TRUE
[2,] TRUE TRUE
[3,] TRUE TRUE
[4,] FALSE TRUE
[5,] TRUE FALSE
```

This also applies to arrays of more than two dimensions.

You can use the `any` and `all` functions to quickly inspect a collection of logical values. When examining a vector, `any` returns `TRUE` if any of the logicals in the vector are `TRUE` and returns `FALSE` otherwise. The function `all` returns `TRUE` only if *all* of the logicals are `TRUE`, and returns `FALSE` otherwise. As a quick example, let's work with two of the logical vectors formed by the comparisons of `foo` and `bar` from the beginning of this section:

```
R> qux <- foo==bar
R> qux
[1] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE TRUE FALSE
R> any(qux)
[1] TRUE
R> all(qux)
[1] FALSE
```

Here, `qux` contains two `TRUE`s, and the rest are `FALSE`, so the result of `any` is `TRUE`, but the result of `all` is `FALSE`. Following the same rules, you get this:

```
R> quux <- foo<=(bar+10)
R> quux
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
R> any(quux)
[1] TRUE
R> all(quux)
[1] TRUE
```

Again, the `any` and `all` functions do the same thing for matrices and arrays of logical values.

Exercise 4.1

- a. Store the following vector of 15 values as an object in your workspace: `c(6,9,7,3,6,7,9,6,3,6,6,7,1,9,1)`. Create corresponding logical-valued vectors that identify the following elements with `TRUE`s:
 - i. Those equal to 6
 - ii. Those greater than or equal to 6
 - iii. Those less than $6 + 2$
 - iv. Those not equal to 6
- b. Create a new numeric vector from the one in (a) by deleting its first three elements. With this new vector, fill a $2 \times 2 \times 3$ array. Create corresponding logical-valued arrays that identify the following elements with `TRUE`s:
 - i. Those less than or equal to 6 divided by 2, plus 4
 - ii. Those less than or equal to 6 divided by 2, plus 4, *after* increasing every element in the array by 2
- c. Confirm with `TRUE`s the specific locations of elements equal to 0 in the 10×10 identity matrix I_{10} (see Section 3.1.3).
- d. Check whether *any* of the values of the logical arrays created in (b) are `TRUE`. If they are, check whether they are *all* `TRUE`.
- e. By extracting the diagonal elements of the logical matrix created in (c), use `any` to confirm there are no `TRUE` entries.

4.1.3 Logical Operators

Logicals are especially useful when examining whether multiple conditions are satisfied. Often you'll want to perform certain operations only if a number of different conditions have been met.

The previous section looked at using relational operators to compare the literal values (that is, numeric or otherwise) of stored R objects. Now you'll look at *logical operators*, which you use to compare two logical-valued objects. These operators are based on the statements `AND` and `OR`. Table 4-2 summarizes the R syntax and the behavior of logical operators. The `AND` and `OR` operators each have a "single" and "element-wise" version; you'll see how they differ in a moment.

Table 4-2: Logical Operators Comparing Two Logical Values

Operator	Interpretation	Results
&	AND (element-wise)	TRUE & TRUE is TRUE
		TRUE & FALSE is FALSE
		FALSE & TRUE is FALSE
		FALSE & FALSE is FALSE
&&	AND (single comparison)	Same as &
	OR (element-wise)	TRUE TRUE is TRUE
		TRUE FALSE is TRUE
		FALSE TRUE is TRUE
		FALSE FALSE is FALSE
	OR (single comparison)	Same as
!	NOT	!TRUE is FALSE
		!FALSE is TRUE

The result of using any logical operator is itself a logical value. An AND comparison is true only if *both* logicals are TRUE. An OR comparison is true if at least one of the logicals is TRUE. The NOT operator (!) returns the opposite of the logical value you use it on. You can combine these operators to examine multiple conditions at once:

```
R> FALSE || ((T&&TRUE) || FALSE)
[1] TRUE
R> !TRUE&&TRUE
[1] FALSE
R> (T&&(TRUE || F))&&FALSE
[1] FALSE
R> (6<4) || (3!=1)
[1] TRUE
```

As with numeric arithmetic, there's an order of importance for logical operations in R: An AND statement has higher precedence than an OR statement. It's helpful to place each comparative pair in parentheses to preserve the correct order of evaluation and make the code more readable. You can see this in the first line of the previous code, where the innermost

comparison is the first to be carried out: `T&&TRUE` results in `TRUE`; this result becomes one of the logical values for the next bracketed comparison where `TRUE||FALSE` results in `TRUE`. The final comparison is then `FALSE||TRUE`, and the end result, `TRUE`, prints to the console. The second line reads as “NOT TRUE AND TRUE,” which returns `FALSE`. In the third line, the innermost pair once again evaluates first: `TRUE||F` is `TRUE`, `T&&TRUE` is `TRUE`, and `TRUE&&FALSE` is `FALSE`. The final example evaluates two distinct conditions in parentheses via relational operators, which you then compare using a logical operator. Since `6<4` is `FALSE` and `3!=1` is `TRUE`, you get a logical comparison of `FALSE||TRUE` and a final result of `TRUE`.

In Table 4-2, there are short (`&`, `|`) and long (`&&`, `||`) versions of the AND and OR operators. Use the short versions in element-wise comparisons, where you have two logical vectors and you want multiple logicals as a result. The long versions, which you’ve been using so far, are for comparing two individual values and return a single logical value. This is important when programming conditional checks in R (for example, with an `if` statement, which you’ll look at in Chapter 10). While it’s possible to compare a single pair of logicals using the short version, it’s considered better practice to use the longer versions when a single `TRUE/FALSE` result is intended.

Let’s look at some examples of element-wise comparisons. Suppose you have two vectors of equal length, `foo` and `bar`:

```
R> foo <- c(T,F,F,F,T,F,T,T,F,T,F)
R> foo
[1] TRUE FALSE FALSE FALSE TRUE FALSE TRUE TRUE TRUE FALSE TRUE FALSE
```

and

```
R> bar <- c(F,T,F,T,F,F,F,T,T,T,T)
R> bar
[1] FALSE TRUE FALSE TRUE FALSE FALSE FALSE TRUE TRUE TRUE TRUE
```

The short versions of the logical operators match each pair of elements by position and return the result of the comparison:

```
R> foo&bar
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE TRUE FALSE
R> foo|bar
[1] TRUE TRUE FALSE TRUE TRUE FALSE TRUE TRUE TRUE TRUE TRUE TRUE
```

Let’s see what happens if you erroneously use the long version of the logical operators to attempt the same multi-element comparison:

```
R> foo&&bar
Error in foo && bar : 'length = 12' in coercion to 'logical(1)'
R> foo||bar
Error in foo || bar : 'length = 12' in coercion to 'logical(1)'
```

As of version 4.3.0+ of R, the interpreter will throw an error when you try to do this; in this case, we're chastised when attempting to do the 12 comparisons comprising the entries of `foo` and `bar`.

Exercise 4.2

- a. Store the vector `c(7,1,7,10,5,9,10,3,10,8)` as `foo`. By constructing a corresponding logical vector with relational and logical operators, identify the elements greater than 5 OR equal to 2.
- b. Store the vector `c(8,8,4,4,5,1,5,6,6,8)` as `bar`. In the same fashion as (a), identify the elements less than or equal to 6 AND not equal to 4.
- c. Identify the elements that satisfy (a) in `foo` AND satisfy (b) in `bar`.
- d. Store a third vector, `baz`, that is equal to the element-wise sum of `foo` and `bar`. Once more using the appropriate operators, construct logical vectors that determine the following:
 - i. The elements of `baz` greater than or equal to 14 but not equal to 15
 - ii. The elements of the vector obtained via an element-wise division of `baz` by `foo` that are greater than 4 OR less than or equal to 2

4.1.4 *Logicals Are Numbers!*

Although this chapter is called “Non-Numeric Values,” logicals (and, as you'll see in subsequent sections, factors and date-time objects) actually have an underlying numeric representation. Because of their binary nature, we often represent logical values in datasets with `TRUE` as 1 and `FALSE` as 0. In fact, when performing elementary numeric operations on logical values, R will actually treat `TRUE` like 1, and `FALSE` like 0:

```
R> TRUE+TRUE
[1] 2
R> FALSE-TRUE
[1] -1
R> T+T+F+T+F+F+T
[1] 4
```

These operations turn out the same as if you'd used the digits 1 and 0. In some situations when using logicals, you can substitute the numeric values (this generally isn't recommended, as it hampers the clarity of your code):

```
R> 1&&1
[1] TRUE
R> 1||0
[1] TRUE
R> 0&&1
[1] FALSE
```

Being able to interpret logicals as 0s and 1s means you can use a variety of functions to summarize a logical vector, which you'll explore further in Part III.

4.1.5 Logical Subsetting and Extraction

You can also use logicals to extract and subset elements in vectors and other objects, in the same way as with index vectors. Rather than entering explicit indexes in the square brackets, you can supply logical *flag* vectors, where an element is extracted if the corresponding entry in the flag vector is TRUE. As such, logical flag vectors and the vector that's being accessed should be the same length (though recycling does occur for shorter flag vectors, as you'll see shortly).

At the beginning of Section 2.3.3 you defined a vector of length 10 as follows:

```
R> myvec <- c(5,-2.3,4,4,4,6,8,10,40221,-8)
```

If you wanted to extract the two negative elements, you could either enter `myvec[c(2, 10)]` or do the following using logical flags:

```
R> myvec[c(F,T,F,F,F,F,F,F,T)]
[1] -2.3 -8.0
```

This particular example may seem far too cumbersome for practical use. The core behavior is incredibly powerful, however, when you implicitly create the desired flag vector based on the desired condition (or several conditions). Taking the current example, remember that you can use logicals to find negative elements in `myvec` by applying the condition `<0`:

```
R> myvec<0
[1] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
```

You can use this flag vector to subset `myvec` to get the same result as earlier. As such, the code

```
R> myvec[myvec<0]
[1] -2.3 -8.0
```

offers up the requested elements without having to explicitly know their specific index positions.

As mentioned, R recycles the flag vector if it's too short. To extract every second element from `myvec`, starting with the first, enter the following:

```
R> myvec[c(T,F)]
[1]    5    4    4    8 40221
```

You can perform more complicated extractions using relational and logical operators, such as:

```
R> myvec[(myvec>0)&(myvec<1000)]
[1]  5  4  4  4  6  8 10
```

This returns the positive elements that are less than 1,000. You can also overwrite specific elements using a logical flag vector, as with index vectors:

```
R> myvec[myvec<0] <- -200
R> myvec
[1]    5 -200    4    4    4    6    8   10 40221 -200
```

This replaces all existing negative entries with `-200`. Note, however, that you can't directly use negative logical flag vectors to delete specific elements; this can be done only with numeric index vectors.

As you can see, logicals are very useful for element extraction. You don't need to know beforehand which index positions to return, since the conditional check can find them for you. This is particularly valuable when you're dealing with large datasets and you want to inspect records or recode entries that match certain criteria.

In some cases, you might want to convert a logical flag vector into a numeric index vector. This is helpful when you *do* need the explicit indexes of elements that were flagged `TRUE`. The R function `which` takes in a logical vector as the argument `x` and returns the indexes corresponding to the positions of any and all `TRUE` entries:

```
R> which(x=c(T,F,F,T,T))
[1] 1 4 5
```

Use `which` to identify the index positions of `myvec` that meet a certain condition; for example, those containing negative numbers:

```
R> which(x=myvec<0)
[1]  2 10
```

You can do this for the other `myvec` selections you experimented with. Note that a line of code such as `myvec[which(x=myvec<0)]` is redundant because you can make that extraction using the condition by itself, that is, via `myvec[myvec<0]`, without using `which`. On the other hand, using `which` lets you *omit* elements based on logical flag vectors by identifying the numeric indexes

you want to delete and rendering them negative. To see what `myvec` looks like without the negative entries, execute the following:

```
R> myvec[-which(x=myvec<0)]
[1]      5      4      4      4      6      8     10 40221
```

The same can be done with matrices and other arrays. In Section 3.1.2, you stored a 3×3 matrix as follows:

```
R> A <- matrix(c(0.3,4.5,55.3,91,0.1,105.5,-4.2,8.2,27.9),nrow=3,ncol=3)
R> A
      [,1] [,2] [,3]
[1,]  0.3  91.0 -4.2
[2,]  4.5   0.1  8.2
[3,] 55.3 105.5 27.9
```

To extract the second and third column elements of the first row of `A` using numeric indexes, execute `A[1,2:3]`. To do the same thing with logical flags, enter the following:

```
R> A[c(T,F,F),c(F,T,T)]
[1] 91.0 -4.2
```

Again, you usually won't explicitly specify the logical vectors. Suppose you want to replace all elements in `A` that are less than 1 with `-7`. Performing this using numeric indexes is rather fiddly. It's much easier to use the logical flag matrix created with the following:

```
R> A<1
      [,1] [,2] [,3]
[1,]  TRUE FALSE  TRUE
[2,]  FALSE  TRUE FALSE
[3,]  FALSE FALSE FALSE
```

You can supply this logical matrix to the square bracket operators. The replacement is done as follows:

```
R> A[A<1] <- -7
R> A
      [,1] [,2] [,3]
[1,] -7.0  91.0 -7.0
[2,]  4.5  -7.0  8.2
[3,] 55.3 105.5 27.9
```

This is the first time you've subsetting a matrix without having to list row or column positions inside the square brackets, using commas to separate out dimensions (see Section 3.1.2). This is because the flag matrix has the same number of rows and columns as the target matrix, thereby providing all the relevant structural information.

If you use `which` to identify numeric indexes based on a logical flag structure, you have to be a little more careful when dealing with two-dimensional

objects or higher. Suppose you want the index positions of the elements that are greater than 25. The appropriate logical matrix is as follows:

```
R> A>25
      [,1] [,2] [,3]
[1,] FALSE TRUE  FALSE
[2,] FALSE FALSE FALSE
[3,]  TRUE  TRUE  TRUE
```

Now ask R the following:

```
R> which(x=A>25)
[1] 3 4 6 9
```

This returns the four indexes of the elements that satisfied the relational check, but they're provided as scalar values. How do these correspond to the row/column positioning of the matrix?

The answer lies in R's default behavior of the `which` function, which essentially treats the multidimensional object as a single vector (laid out column after column) and then returns the vector of corresponding indexes. If you explicitly arrange the matrix `A` as a vector by stacking the columns first through third, using `c(A[,1],A[,2],A[,3])`, then the indexes returned make more sense:

```
R> which(x=c(A[,1],A[,2],A[,3])>25)
[1] 3 4 6 9
```

With the columns laid out end to end, the elements that return `TRUE` are the third, fourth, sixth, and ninth elements in the list. This can be difficult to interpret, though, especially when dealing with higher-dimensional arrays. In these situations, make `which` return dimension-specific indexes using the optional argument `arr.ind` ("array indexes"). By default, this argument is set to `FALSE`, resulting in the vector converted indexes. Setting `arr.ind` to `TRUE`, on the other hand, treats the object as a matrix or array rather than a vector, providing the row and column positions of the elements you requested.

```
R> which(x=A>25,arr.ind=TRUE)
      row col
[1,]   3   1
[2,]   1   2
[3,]   3   2
[4,]   3   3
```

The returned object is now a matrix, where each row represents an element that satisfied the logical comparison and each column provides the position of the element. Comparing the output here with `A`, these positions correspond to elements where `A>25`.

Both versions of the output (with `arr.ind=TRUE` or `arr.ind=FALSE`) are useful, depending on the application.

Exercise 4.3

- a. Store this vector of 10 values: `foo <- c(7,5,6,1,2,10,8,3,8,2)`. Then, do the following:
 - i. Extract the elements greater than or equal to 5, storing the result as `bar`.
 - ii. Display the vector containing those elements from `foo` that remain after omitting all elements that are greater than or equal to 5.
- b. Use `bar` from (a)(i) to construct a 2×3 matrix called `baz`, filled in a row-wise fashion. Then, do the following:
 - i. Replace any elements that are equal to 8 with the *squared* value of the element in row 1, column 2 of `baz` itself.
 - ii. Confirm that *all* values in `baz` are now less than or equal to 25 AND greater than 4.
- c. Create a $3 \times 2 \times 3$ array called `qux` using the following vector of 18 values: `c(10,5,1,4,7,4,3,3,1,3,4,3,1,7,8,3,7,3)`. Then, do the following:
 - i. Identify the dimension-specific index positions of elements that are either 3 OR 4.
 - ii. Replace all elements in `qux` that are less than 3 OR greater than or equal to 7 with the value 100.
- d. Return to `foo` from (a). Use the vector `c(F,T)` to extract every second value from `foo`. In some situations, you can substitute 0 and 1 for TRUE and FALSE (see Section 4.1.4). Can you perform the same extraction from `foo` using the vector `c(0,1)`? Why/why not? What does R return in this case?

4.2 Characters

Character strings are a common data type we use to represent text. In R, we often use strings to specify folder locations or software options (see the brief mention in Section 1.2); to supply an argument to a function; and to annotate stored objects, provide textual output, or help clarify plots and graphics. While we can also use strings to define different groups making up a categorical variable, as you'll see in Section 4.3, *factors* are specifically designed (and hence better suited) for that.

NOTE

There are three different string formats in the R environment. The default string format is an extended regular expression; the other variants are Perl and literal regular expressions. The intricacies of these variants are beyond the scope of this book, so any mention of character strings from here on refers to an extended regular expression. For technical details about other string formats, enter `?regex` at the prompt.

4.2.1 String Creation

We indicate character strings with double quotation marks ("). To create a string, enter text between a pair of quotes:

```
R> foo <- "This is a character string!"
R> foo
[1] "This is a character string!"
R> length(x=foo)
[1] 1
```

R treats the string as a single entity. In other words, `foo` is a vector of length 1 because R counts only the total number of distinct strings rather than individual words or characters. To count the number of individual characters, use the `nchar` function. Here's an example using `foo`:

```
R> nchar(x=foo)
[1] 27
```

Almost any combination of characters, including numbers, can be a valid character string, as shown here:

```
R> bar <- "23.3"
R> bar
[1] "23.3"
```

In this form, the string has no numeric meaning, and R won't treat it like the number 23.3. Attempting to multiply it by 2, for example, results in an error:

```
R> bar*2
Error in bar * 2 : non-numeric argument to binary operator
```

This error occurs because `*` is expecting to operate on two numeric values (not one number and one string, which makes no sense).

You can compare strings in several ways, the most common being a check for equality:

```
R> "alpha"=="alpha"
[1] TRUE
R> "alpha"!="beta"
[1] TRUE
R> c("alpha", "beta", "gamma")=="beta"
[1] FALSE TRUE FALSE
```

Other relational operators work as you might expect. For example, R considers letters that come later in the alphabet to be greater than earlier letters, meaning it can determine whether one string of letters is greater than another with respect to alphabetical order:

```
R> "alpha"<="beta"
[1] TRUE
```

```
R> "gamma">"Alpha"
[1] TRUE
```

Furthermore, R treats uppercase letters as “greater” than lowercase letters:

```
R> "Alpha">"alpha"
[1] TRUE
R> "beta">"bEtA"
[1] FALSE
```

You can use most symbols in a string. The following string is valid, for example:

```
R> baz <- "& _ 3 **%.? $symbolic non$en$e ,; "
R> baz
[1] "& _ 3 **%.? $symbolic non$en$e ,; "
```

One important exception is the backslash `\`, or *escape*. Using a backslash within the quotation marks of a string initiates some simple control over the printing or display of the string itself. You’ll see how this works in Section 4.2.3. First, let’s look at two useful functions for combining strings.

4.2.2 Concatenation

There are two main functions we use to *concatenate* (or glue together) one or more strings: `cat` and `paste`. The difference between the two lies in how their contents are returned. The first function, `cat`, sends its output directly to the console screen and doesn’t formally *return* anything. The `paste` function concatenates its contents and then returns the final character string as a usable R object. This is useful when R needs to pass the result of a string concatenation to another function or use it in some secondary way, as opposed to just displaying it. Consider the following vector of character strings:

```
R> qux <- c("awesome", "R", "is")
R> length(x=qux)
[1] 3
R> qux
[1] "awesome" "R"      "is"
```

As with numbers and logicals, you can also store any number of strings in a matrix or array structure.

When calling `cat` or `paste`, you pass arguments to the function in the order you want them combined. The following lines show identical usage yet different types of output from the two functions:

```
R> cat(qux[2],qux[3],"totally",qux[1],"!")
R is totally awesome !
R> paste(qux[2],qux[3],"totally",qux[1],"!")
[1] "R is totally awesome !"
```

Here, you've used the three elements of `qux` as well as two additional strings, "totally" and "!", to produce the final concatenated string. In the output, `cat` simply concatenates and prints the text to the screen. This means you can't directly assign the result to a new variable and treat it as a character string. For `paste`, however, the `[1]` to the left of the output and the presence of the double quotes indicate the returned item is a vector containing a character string, which you can assign to an object and use in other functions.

These two functions have an optional argument, `sep`, that's used as a separator between strings as they're concatenated. If you pass `sep` a character string, it places this string between all other strings you've provided to `paste` or `cat`. For example:

```
R> paste(qux[2],qux[3],"totally",qux[1],"!",sep="---")
[1] "R---is---totally---awesome---!"
R> paste(qux[2],qux[3],"totally",qux[1],"!",sep="")
[1] "Ristotallyawesome!"
```

The same behavior would occur for `cat`. If you don't want any separation, set `sep=""`, an empty string, as in the second example. You can use the empty string separator to achieve correct sentence spacing; note the gap between `awesome` and the exclamation mark in the code where you first used `paste` and `cat`. If the `sep` argument isn't included, R inserts a space between strings by default. For example, using manual insertion of spaces where necessary, write the following:

```
R> cat("Do you think ",qux[2]," ",qux[3]," ",qux[1],"?",sep="")
Do you think R is awesome?
```

Concatenation can be useful when you want to neatly summarize the results from a certain function or set of calculations. You can pass many R objects directly to `paste` or `cat`; the software will attempt to automatically *coerce* these items into character strings. This means R converts the input into a string to include the values in the final concatenated string. This works particularly well with numeric objects, as the following examples demonstrate:

```
R> aa <- 3
R> bb <- 4.4
R> cc <- 10
R> cat("The value stored as 'aa' is ",aa,".",sep="")
The value stored as 'aa' is 3.
R> paste("The value stored as 'bb' is ",bb,".",sep="")
[1] "The value stored as 'bb' is 4.4."
R> cat("The result of aa+bb is ",aa,"+",bb,"=",aa+bb,".",sep="")
The result of aa+bb is 3+4.4=7.4.
R> paste("Is ",aa+bb," less than ",cc,"? That's totally ",aa+bb<cc,".",sep="")
[1] "Is 7.4 less than 10? That's totally TRUE."
```

Here, the values of the non-string objects are placed where you want them in the final string output. The results of calculations can also appear as fields, as shown with the arithmetic `aa+bb` and the relational comparison `aa+bb<cc`. You'll see more details about coercion from one kind of value to another in Section 6.2.4.

The function `paste0` is identical to `paste` in behavior but has `sep=""` by default. If you want a no-space separator between your strings, use this variant of the function. Repeating the most recent example, see that

```
R> paste0("Is ",aa+bb," less than ",cc,"? That's totally ",aa+bb<cc, ".")
[1] "Is 7.4 less than 10? That's totally TRUE."
```

didn't require you to explicitly set `sep=""`.

It's worth highlighting that the same kind of vectorized behavior in terms of element-wise operations (refer to Section 2.3.4) also applies to `paste` (and `paste0`). Consider the following vector of eight character strings:

```
R> poetry <- c("Hey","diddle","diddle","the","cat","and","the","fiddle")
R> length(poetry)
[1] 8
```

We can easily augment each of these components in the same way using vector-oriented behavior. For example:

```
R> paste(poetry,"!")
[1] "Hey !"      "diddle !"   "diddle !"   "the !"      "cat !"      "and !"      "the !"
[8] "fiddle !"
```

Here, an exclamation mark (with a default space separator given our use of `paste` instead of `paste0`) simply pastes onto the end of each string in `poetry`. Provided evenly divisible lengths (as Figure 2-2 illustrates), recycling of the shorter vector occurs in the typical way:

```
R> paste0(poetry,c("?", "!"))
[1] "Hey?"      "diddle!"   "diddle?"   "the!"      "cat?"      "and!"      "the?"
[8] "fiddle!"
```

The two components to be pasted onto the elements of `poetry`, `"?"` and `"!"`, replicate four times to match the longer vector's length prior to pasting (with no space this time, due to use of `paste0`).

Lastly, it's also possible to concatenate an existing vector of multiple character strings together into a single character string with `paste`, though you need to make use of an optional argument. Remaining with the eight components in `poetry`, suppose you want to 'glue' them all together. Simply calling `paste` on that alone doesn't do anything, leaving the existing vector untouched:

```
R> paste(poetry)
[1] "Hey"      "diddle"   "diddle"   "the"      "cat"      "and"      "the"      "fiddle"
```

To achieve the desired outcome, use the `collapse` argument, which specifies the character string used to separate the multiple elements of the target vector when they're appended to one another. The following three examples demonstrate this behavior:

```
R> paste(poetry,collapse=" ")
[1] "Hey diddle diddle the cat and the fiddle"
R> paste(poetry,collapse="")
[1] "Heydiddlediddlethecatandthefiddle"
R> paste(poetry,collapse="&-%")
[1] "Hey&-%diddle&-%diddle&-%the&-%cat&-%and&-%the&-%fiddle"
```

4.2.3 Escape Sequences

In Section 4.2.1, I noted that a stand-alone backslash doesn't act like a normal character within a string. Using `\` invokes an *escape sequence*, which lets you enter characters that control the format and spacing of the string, rather than being interpreted as normal text. Table 4-3 describes some of the most common escape sequences; for a full list, enter `?Quotes` at the prompt.

Table 4-3: Common Escape Sequences for Use in Character Strings

Escape sequence	Result
<code>\n</code>	Starts a newline
<code>\t</code>	Horizontal tab
<code>\\</code>	Used as a single backslash
<code>\"</code>	Includes a double quote

Escape sequences add flexibility to the display of character strings, which can be useful for summaries of results and plot annotations. Enter the sequence precisely where you want it to take effect; for example:

```
R> cat("here is a string\nsplit\tto new\n\n\tlines")
here is a string
split to new

lines
```

Note that the exact amount of space imparted by a single use of the tab escape, `\t`, depends on the environment and context in which you're using R. For example, using the base R GUI exhibits a different default tab spacing than the RStudio IDE in macOS, and so the specific appearance of such output can differ between systems.

The signal for an escape is `\` and the signal to begin and end a string is `"`, meaning if you want to include either character in a string, you must use an escape to have R interpret them as a normal character:

```
R> cat("I really want a backslash: \\nand a double quote: \")
I really want a backslash: \
and a double quote: "
```

These escape sequences are the reason you can't use a stand-alone backslash in filepath strings in R. As noted in Section 1.3.4 (where you used `getwd` to print the current working directory and `setwd` to change it), folder separation is best designated with a single *forward* slash (`/`). Filepath specification is directly relevant when reading in and writing out files, and you'll explore these activities in greater detail in Chapter 8.

4.2.4 Substrings and Matching

Pattern matching lets you inspect a given string to identify smaller “query” strings within it. For the simplest form of pattern matching, use the `grep` function to inspect a vector of character strings and return the indexes (if any) of the elements that contain the query string. For example:

```
R> poetry <- c("Hey", "diddle", "diddle", "the", "cat", "and", "the", "fiddle")
R> result1 <- grep(pattern="iddle", x=poetry)
R> result1
[1] 2 3 8
```

Here, you apply `grep` to the vector of character strings in the `poetry` object from Section 4.2.2. Via the `pattern` argument, you're asking for matches to the query string “iddle”. The result, stored as `result1`, is a numeric vector containing 2, 3, and 8—correctly identifying the positions of those strings in `poetry` that contain the pattern of interest. When such a search yields no results, you see the following:

```
R> result2 <- grep(pattern="dog", x=poetry)
R> result2
integer(0)
```

The somewhat curious notation `integer(0)` simply denotes an “empty” vector, evidenced by use of `length` on the object:

```
R> length(x=result2)
[1] 0
```

This is fitting, seeing as there are no strings in `poetry` containing the substring “dog”.

A related function is `substr`, which takes a target string `x` and returns the substring that sits between two character positions, indicated with numbers

passed to the start and stop arguments. Let's try it on the object `foo` from Section 4.2.1:

```
R> foo <- "This is a character string!"
R> substr(x=foo,start=21,stop=27)
[1] "string!"
```

Here, you extract the characters between (and including) positions 21 and 27, to get "string!". The function also works on vectors of character strings; the following example returns the first three characters of each entry in `poetry`:

```
R> substr(x=poetry,start=1,stop=3)
[1] "Hey" "did" "did" "the" "cat" "and" "the" "fid"
```

You can also use the function `substr` with the assignment operator to directly substitute in a new set of characters. In this case, the replacement string should contain the same number of characters as the selected area:

```
R> substr(x=foo,start=1,stop=4) <- "Here"
R> foo
[1] "Here is a character string!"
```

If the replacement string is longer than the number of characters indicated by start and stop, then replacement still takes place, beginning at start and ending at stop. It cuts off any characters that overrun the number of characters you're replacing. If the string is shorter than the number of characters you're replacing, then replacement ends when the string is fully inserted, leaving the original characters up to stop untouched.

Other tools for substitution include the functions `sub` and `gsub`. The `sub` function searches a given string `x` for a smaller string pattern contained within. It then replaces the first instance with a new string, given as the replacement argument. The `gsub` function does the same thing, but it replaces *every* instance of pattern. Here's an example:

```
R> bar <- "How much wood could a woodchuck chuck"
R> sub(pattern="chuck",replacement="hurl",x=bar)
[1] "How much wood could a woodhurl chuck"
R> gsub(pattern="chuck",replacement="hurl",x=bar)
[1] "How much wood could a woodhurl hurl"
```

Note the replacement value doesn't need to have the same number of characters as the pattern being replaced.

The help files `?grep` and `?substr` (the former also documents `sub` and `gsub`) have more details, as well as noting a handful of other pattern-matching functions and techniques. You can disable case sensitivity, which is the default when searching for substrings with functions like `grep` and `gsub`.

Exercise 4.4

- a. By using `cat` on a string with the appropriate escape sequences, re-create the following output:

```
"The quick brown fox
      jumped over
      the lazy dogs"
```

- b. Suppose you've stored the values `num1 <- 4` and `num2 <- 0.75`. Write a line of R code that returns the following string:

```
[1] "The result of multiplying 4 by 0.75 is 3"
```

Make sure your code produces a string with the correct multiplication result for *any* two numbers stored as `num1` and `num2`.

- c. On my local machine, I've specified the directory for my work on this book in R as `"/Users/tdavies/Documents/Book"`. Imagine it is your machine, and write a line of code that replaces `tdavies` in this string with your own name.
- d. Using the appropriate R function, determine which entries of the character string vector given by

```
cutlery <- c("the", "dish", "ran", "away", "with", "the", "spoon")
```

contain the substring `"th"`.

- e. In Section 4.2.4, you stored the following string:

```
bar <- "How much wood could a woodchuck chuck"
```

- i. Store a new string by gluing the following words onto `bar`: `"if a woodchuck could chuck wood"`.
 - ii. In the result of (i), replace all instances of `wood` with `metal`.
- f. Store the string `"Two 6-packs for $15.99"`, and then do the following:
- i. Use `substr` and a check for equality to confirm that the substring beginning with character 5 and ending with character 10 is `"6-pack"`.
 - ii. Make it a better deal by using `substr` to change the price to `$12.99`.

4.3 Factors

In this section, you'll look at some simple functions related to creating, handling, and inspecting *factors*. Factors are R's most natural way of representing data values which take on only one of a finite number of distinct categories, rather than belonging to a continuum. Categorical data can play an important role in data science, and you'll look at factors again from a statistical perspective in Chapter 13.

4.3.1 Identifying Categories

To see how factors work, we'll start with a simple dataset. Suppose you find eight people and record their first name, sex, and birth month in Table 4-4.

Table 4-4: An Example Dataset of Eight Individuals

Person	Sex	Birth month
Liz	Female	April
Jolene	Female	January
Susan	Female	December
Boris	Male	September
Rochelle	Female	November
Tim	Male	July
Simon	Male	July
Amy	Female	June

There's really only one sensible way to represent the name of each person in R, which is as a vector of character strings:

```
R> firstname <- c("Liz", "Jolene", "Susan", "Boris", "Rochelle", "Tim", "Simon",
                  "Amy")
```

You have more flexibility when recording sex, however. Coding females as 0 and males as 1, a numeric option would be as follows:

```
R> sex.num <- c(0,0,0,1,0,1,1,0)
```

Character strings are also possible, and many prefer this because you don't need to remember the numeric code for each group, as shown here:

```
R> sex.char <- c("female", "female", "female", "male", "female", "male", "male",
                "female")
```

There is, however, a fundamental difference between an individual's name and their sex when stored as data. Where a person's name is a unique identifier that can take any one of an infinite number of possibilities, there

are typically only two options for recording a person's sex. It's best to represent this kind of variable, wherein all possible values fall into a finite number of categories, as a factor in R.

You typically create factors from a numeric or a character vector. You can't fill matrices or multidimensional arrays using factor values; factors can take only the form of vectors. To create a factor, use the function `factor`, as in this example working with `sex.num` and `sex.char`:

```
R> sex.num.fac <- factor(x=sex.num)
R> sex.num.fac
[1] 0 0 0 1 0 1 1 0
Levels: 0 1
R> sex.char.fac <- factor(x=sex.char)
R> sex.char.fac
[1] female female female male   female male   male   female
Levels: female male
```

Here, you obtain factor versions of the two vectors storing the sex values.

At first glance, these objects don't look too different from the character and numeric vectors from which they were created. Indeed, factor objects work in much the same way as vectors, but with extra information attached (R's internal representation of factor objects is a bit different as well). Simple functions like `length` and `which` work the same way on factor objects as with vectors, for example.

The most important extra piece of information (or *attribute*; see Section 6.2.1) that a factor object contains is its *levels* (or *categories*), which define the possible values of the factor. R prints these levels at the bottom of each factor vector. To extract the levels as a vector of character strings, use the `levels` function:

```
R> levels(x=sex.num.fac)
[1] "0" "1"
R> levels(x=sex.char.fac)
[1] "female" "male"
```

You can also relabel a factor using `levels`. Here's an example:

```
R> levels(x=sex.num.fac) <- c("1","2")
R> sex.num.fac
[1] 1 1 1 2 1 2 2 1
Levels: 1 2
```

This relabels the females 1 and the males 2.

Factors are subsetted in the same way as any other vector:

```
R> sex.char.fac[2:5]
[1] female female male   female
Levels: female male
R> sex.char.fac[c(1:3,5,8)]
```

```
[1] female female female female female
Levels: female male
```

After subsetting a factor object, the object continues to store *all* defined levels, even if the subsetted object no longer represents some levels.

If you want to subset from a factor using a logical flag vector, keep in mind that R recognizes the levels of a factor as character strings, even if the original data vector was numeric, so you need to use a string when requesting or testing for a particular level. For example, to identify all the men using the newly relabeled `sex.num.fac`, use this:

```
R> sex.num.fac=="2"
[1] FALSE FALSE FALSE  TRUE FALSE  TRUE  TRUE FALSE
```

Since the elements in `firstname` and `sex` have corresponding positions, you can then use a logical vector like the previous one to obtain the names of all the men (this time using the "male"/"female" factor):

```
R> firstname[sex.char.fac=="male"]
[1] "Boris" "Tim"  "Simon"
```

Of course, this simple subsetting operation could have been achieved in much the same way with the raw numeric vector `sex.num` or the raw character vector `sex.char`. In the next section, you'll explore distinctive advantages to R representing categorical data as a factor.

4.3.2 Defining and Ordering Levels

The sex factor from the previous section represents the simplest kind of factor variable; there are only two possible levels with no ordering, in that one level is not intuitively considered "higher than" or "following" the other. Here, you'll look at factors with more than two levels, as well as defining and customizing levels in greater detail.

To motivate things, we'll focus on the month of birth (MOB) data for the eight individuals in Table 4-4, first storing the corresponding entries as a simple character vector:

```
R> mob <- c("Apr", "Jan", "Dec", "Sep", "Nov", "Jul", "Jul", "Jun")
```

Like `sex`, it makes sense to treat MOB as a categorical variable. If we use `factor` in exactly the same way as before, we get something like this:

```
R> mob.fac.basic <- factor(x=mob)
R> mob.fac.basic
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels: Apr Dec Jan Jul Jun Nov Sep
```

There are two potential problems with this basic version of the MOB factor. First, not all possible categories are represented because the original `mob` character vector contains only seven unique months. Tabulating the data

with the function `table`, we see two observations categorized as “July,” with the remaining months represented only once:

```
R> table(mob.fac.basic)
mob.fac.basic
Apr Dec Jan Jul Jun Nov Sep
  1   1   1   2   1   1   1
```

Second, the levels of the factor aren’t arranged in the “natural” calendar order of months; the default behavior in R is to sort alphabetically.

While these two issues may seem minor in isolation, they can have important consequences when wrangling data and using categorical variables in statistical analyses. You can deal with both problems to varying extents by customizing additional arguments to the factor function. The most important of these is the `levels` argument, which takes a character vector defining all the possible categories you want, in the order that you want. Redefine the MOB factor as follows:

```
R> ms <- c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov",
           "Dec")
R> mob.fac <- factor(x=mob, levels=ms)
R> mob.fac
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Here, R defines the `mob.fac` vector in terms of the same eight individual entries (the `mob` object) as earlier. The new factor has 12 levels due to using `levels=ms` inside the call to `factor`, even though you haven’t actually made observations for the levels “Feb”, “Mar”, “May”, “Aug”, or “Oct”. (Note that if your R console window is too narrow to print all the levels to the screen, you may see `...`, indicating there’s more output. Widen your window and reprint the object to see the hidden levels.) In addition, R arranges the new category levels in the order you supplied them to the `levels` argument. Once more using the `table` function, you obtain the following:

```
R> table(mob.fac)
mob.fac
Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
  1   0   0   1   0   1   2   0   1   0   1   1
```

NOTE

You can impose a stricter, more formal ordering on a factor by setting the optional argument `ordered=TRUE` when using the `factor` function (the default has this at `FALSE`, which is how I’ve used `factor` in the preceding code). Doing this, however, can have important consequences in the implementation of various statistical methods, such as regression and other types of modeling: It changes R’s handling and interpretation of categorical variables. As such, it’s used only in very specific circumstances and is generally safe to ignore. See the help file `?factor` for further details on these more complicated features of factors.

The most direct way to re-order the levels of a given factor after the fact is to apply the factor function to the existing factor with the preferred order appearing in the levels argument. To illustrate this, re-create the `sex.char.fac` factor from Section 4.3.1 here:

```
R> sex.char <- c("female", "female", "female", "male", "female", "male", "male",
               "female")
R> sex.char.fac <- factor(x=sex.char)
R> sex.char.fac
[1] female female female male   female male   male   female
Levels: female male
```

To arrange the categories the other way around, with males first, do the following:

```
R> sex.char.fac2 <- factor(x=sex.char.fac, levels=c("male", "female"))
R> sex.char.fac2
[1] female female female male   female male   male   female
Levels: male female
```

The individual entries are entirely unchanged; only the order of the level definitions are modified.

The `relevel` function is useful if you just want to ensure a particular level comes first (we sometimes refer to this as a *reference category* or *reference level*). As the sex factor has only two levels to begin with, you could execute the following line to equivalently create `sex.char.fac2`:

```
R> relevel(x=sex.char.fac, ref="male")
[1] female female female male   female male   male   female
Levels: male female
```

Note that you supply the desired reference category as a character string to the `ref` argument, and you must take care to ensure its value matches exactly (including case sensitivity) one of the existing levels of the target factor.

You can see that `relevel` changes only the category that comes first by operating on a factor with more than two levels. Using `mob.fac` from earlier, consider the following:

```
R> relevel(x=mob.fac, ref="Dec")
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels: Dec Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov
```

Here, you ask that December becomes the first category; all other levels remain in the same order as originally defined when `mob.fac` was created. (Note also that you haven't actually changed `mob.fac` or `sex.char.fac` as they're stored; to do that, you need to overwrite/reassign the objects with their relevelled selves.) The reference level or "first" category of a factor affects the treatment of categorical variables in some situations (you'll see examples of this in Chapter 21's discussion of multiple regression models).

4.3.3 Dropping Categories

Remember that changes to the levels of a factor, both in terms of the available categories and in terms of their ordering, are far from just cosmetic. There's a big difference, for example, between a dataset with zero observations in some of the categories and the same dataset defined with fewer categories to begin with. That said, you may occasionally need to simplify a factor by removing categories that aren't represented by the observation set at hand. This typically occurs after you've subsetting a larger dataset and are interested only in populated categories.

Consider again the `mob.fac` object defined in the previous section. You created this specifically to include levels that weren't represented by the entries in the `mob` vector; that is, they were associated with "zero counts" when you tabulated the factor. The object is reprinted here:

```
R> mob.fac
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Suppose you're given this factor but are interested only in analyzing the categories represented by the data. The `droplevels` function creates a new factor with the exact same data but strips away any categories with zero counts:

```
R> mob.fac2 <- droplevels(x=mob.fac)
R> mob.fac2
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels: Jan Apr Jun Jul Sep Nov Dec
```

Notice that the remaining categories of the new `mob.fac2` are precisely the same as those of the `mob.fac.basic` vector created at the very beginning of Section 4.3.2, but their order is different (retaining the order imposed by `mob.fac`).

4.3.4 Working with the Numeric Representation of Factors

Just like logical-valued vectors, all factors have an equivalent numeric (integer) form that essentially designates the category number to which a given observation belongs. Category numbers can be useful when using R for things like data visualization.

To see the integer form of any factor, use the `as.numeric` function directly on the object of interest. Return once more to the `sex.char.fac` factor as the preceding sections show:

```
R> sex.char.fac
[1] female female female male   female male   male   female
Levels: female male
```

Its numeric form is as follows:

```
R> as.numeric(sex.char.fac)
[1] 1 1 1 2 1 2 2 1
```

You're returned a vector of integers with the same number of elements as the original factor, identifying the category each entry belongs to. The output tells you that the first three individuals are in category 1, the fourth in category 2, the fifth in category 1, and so on. The numbering of the categories always follows the defined order of the factor levels; in this example, the numeric form agrees with the displayed order of the factor levels in `sex.char.fac`, in that females are identified as category 1, and males as 2.

If the defined order of the levels changes, so too do the numeric identifiers of each entry. Taking the `sex.char.fac2` object from Section 4.3.2 (in which the males are now defined as the first category), observe the consistency of the numeric form of the factor:

```
R> sex.char.fac2
[1] female female female male   female male   male   female
Levels: male female
R> as.numeric(sex.char.fac2)
[1] 2 2 2 1 2 1 1 2
```

Again, while the individual entries of `sex.num.fac2` are exactly the same, their numeric representation is now different purely by virtue of the re-ordered factor levels.

The `as.numeric` function is one of a collection of so-called “as-dot” functions we use to convert between different object and data types (known formally as *coercion*). We'll study its behavior in greater detail in Section 6.2.4.

4.3.5 Collapsing Factors

In some situations, you may be dealing with factors that have many levels and, for one reason or another, you wish to collapse the factor to have fewer levels. For example, you may have several categories with very low counts and, for the purposes of your analysis, seek to simplify matters by collecting together these rarer levels into some overall “Other” category.

To illustrate this, we'll use the `mob.fac` object created in Section 4.3.2 on page 84, detailing the month of birth of eight individuals (Table 4-4). As a reminder, here is the form of the object and the tabulated counts:

```
R> mob.fac
[1] Apr Jan Dec Sep Nov Jul Jul Jun
Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
R> table(mob.fac)
mob.fac
Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
  1   0   0   1   0   1   2   0   1   0   1   1
```

Suppose we wish to collapse this object so that the unique categories are not the individual months, but instead are coded as quarters. That is, we wish to create a new object whereby all individuals with a birth month of January, February, or March are assigned the same category, *Q1*; those with a birth month of either April, May, or June are categorized as *Q2*, and so on. One way to do this would be to convert the factor to its numeric representation (as per Section 4.3.4), recognize the fact that numbers 1 through 3 would correspond to *Q1*, 4 through 6 would be *Q2*, and so on, and use this to construct a new factor from scratch with the `factor` function. This is a lot of work! Instead, we can use the existing factor and simply re-define the levels themselves. The definition and membership of the new, collapsed categories must be supplied in the form of a *named list*, which is a construct for collecting together multiple objects. You've not yet met lists; these will be scrutinized in the following chapter (Section 5.1).

For now, consider the following syntax, which first creates an identical copy of `mob.fac` (named `mob.fac.q`) and subsequently proceeds to manipulate the categories by changing the representation of the levels in a particular way:

```
R> mob.fac.q <- mob.fac
R> levels(x=mob.fac.q) <- list(Q1=c("Jan", "Feb", "Mar"), Q2=c("Apr", "May", "Jun"),
                               Q3=c("Jul", "Aug", "Sep"), Q4=c("Oct", "Nov", "Dec"))
```

Note that the key to this action involves overwriting (or assigning) the `levels` attribute of the target factor object with the desired definition. In a call to `list`, you collect the labels of the pre-existing factor that you want to collapse together as a vector of character strings, and assign each vector to a tag that will become the new label (the latter needn't be supplied in double quotes). In this example, you can see I intend to re-code my factor `mob.fac.q` to be made up of four unique levels, labeled *Q1*, *Q2*, *Q3*, and *Q4*. The new category *Q1* will be given to any of the original observations that correspond to January, February, or March, while *Q2* will represent any entries corresponding to April, May, or June, and so on. You must ensure the character string labels you're collecting together match the levels of the original factor exactly.

Inspecting and tabulating the newly re-coded object `mob.fac.q`, we see the following:

```
R> mob.fac.q
[1] Q2 Q1 Q4 Q3 Q4 Q3 Q3 Q2
Levels: Q1 Q2 Q3 Q4
R> table(mob.fac.q)
mob.fac.q
Q1 Q2 Q3 Q4
 1  2  3  2
```

You can compare the individual entries of `mob.fac.q` with those of the original `mob.fac` to ensure things correspond as expected. For instance, the first entry of `mob.fac.q` is *Q2*; this makes perfect sense because the first entry of `mob.fac` is *Apr*. Tabulation further confirms things have worked correctly; there are exactly one, two, three, and two individual months that fall in *Q1*, *Q2*, *Q3*, and *Q4* of the calendar year, respectively.

4.3.6 Combining Factors

As you’ve seen, it’s usually simple to combine multiple vectors of the same kind (whether numeric, logical, or character) using the `c` function. In R versions $\geq 4.1.0$, you can do the same thing. As a quick toy example, consider the following two factor objects, each of which has four elements:

```
R> foo <- factor(x=c("A","B","A","C"))
R> foo
[1] A B A C
Levels: A B C
R> bar <- factor(x=c("D","E","E","E"))
R> bar
[1] D E E E
Levels: D E
```

Note that `foo` and `bar` are completely different objects; the former has three levels labeled "A", "B" and "C", and the latter has only two possible categories, "D" and "E". Create a new factor by combining everything via `c`:

```
R> baz <- c(foo,bar)
R> baz
[1] A B A C D E E E
Levels: A B C D E
```

Here, the new factor has eight observations spanning five possible categories.

More realistically, you may simply want to add newly obtained observations to an existing object, where the new observations take on categories the existing object has already defined. Let’s see this behavior on the data in Table 4-4; specifically, the MOB factor vector `mob.fac`, from Section 4.3.2. Suppose you now observe three more individuals with MOB values "Oct", "Feb", and "Feb". You can quickly create something like

```
R> new.mob.values <- factor(x=c("Oct","Feb","Feb"))
R> new.mob.values
[1] Oct Feb Feb
Levels: Feb Oct
```

and append them to the existing observations of `mob.fac`:

```
R> mob.fac.ext <- c(mob.fac,new.mob.values)
R> mob.fac.ext
[1] Apr Jan Dec Sep Nov Jul Jul Jun Oct Feb Feb
Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

The three new entries now appear at the end of the eight original values. Furthermore, the fact that the levels of `new.mob.values` (only “Feb” and “Oct”) aren’t exactly equivalent to the entire set of 12 possible levels of `mob.fac` doesn’t matter; R recognizes that `Feb` and `Oct` already exist as category labels in `mob.fac`.

This behavior means you must take care with things that might be missed, such as case sensitivity. For example, if you accidentally misspell a given entry, R interprets this as a new category, as illustrated here:

```
R> bad.mob.values <- factor(x=c("Oct", "Feb", "FeB"))
R> c(mob.fac, bad.mob.values)
[1] Apr Jan Dec Sep Nov Jul Jul Jun Oct Feb FeB
Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec FeB
```

This is something you'd be keen to avoid in practice. While such a mistake is easily visible in a small example, in situations with hundreds or thousands of observations, attention to detail is key.

4.3.7 Creating Factors from Continuous Values

You often create factors from data originally measured on a continuum, for example the weight of a set of adults or the amount of a drug given to a patient. Sometimes you need to group (or *bin*) these types of observations into categories, like Small/Medium/Large or Low/High. In R, you can mold these kinds of data into discrete factor categories using the `cut` function. Consider the following numeric vector of length 10:

```
R> Y <- c(0.53, 5.4, 1.5, 3.33, 0.45, 0.01, 2, 4.2, 1.99, 1.01)
```

Suppose you want to bin the data as follows: *Small* refers to observations in the interval $[0, 2)$, *Medium* refers to $[2, 4)$, and *Large* refers to $[4, 6]$. A square bracket refers to *inclusion* of its nearest value, and a parenthesis indicates *exclusion*, so an observation y falls in the Small interval if $0 \leq y < 2$, in Medium if $2 \leq y < 4$, or in Large if $4 \leq y \leq 6$. Use `cut` and supply your desired break intervals to the `breaks` argument:

```
R> br <- c(0, 2, 4, 6)
R> cut(x=Y, breaks=br)
[1] (0,2] (4,6] (0,2] (2,4] (0,2] (0,2] (0,2] (4,6] (0,2] (0,2]
Levels: (0,2] (2,4] (4,6]
```

This gives you a factor, with each observation now assigned an interval. However, notice that the boundary intervals are back-to-front; you want the boundary levels on the left like $[0, 2)$, rather than the right as they appear by default, $(0, 2]$. Fix this by setting the `logical` argument `right` to `FALSE`:

```
R> cut(x=Y, breaks=br, right=FALSE)
[1] [0,2) [4,6) [0,2) [2,4) [0,2) [0,2) [2,4) [4,6) [0,2) [0,2)
Levels: [0,2) [2,4) [4,6)
```

Now you've swapped which boundaries are inclusive and exclusive. This is important because it changes which categories the values fall into. Notice that the seventh observation has changed categories. But there's still a problem: The final interval currently *excludes* 6, and you want this maximum value to be *included* in the highest level. You can fix this with another logical

argument, `include.lowest`, which you also use to include the highest value if `right` is `FALSE`, as the `?cut` help file indicates:

```
R> cut(x=Y,breaks=br,right=FALSE,include.lowest=TRUE)
[1] [0,2) [4,6] [0,2) [2,4) [0,2) [0,2) [2,4) [4,6] [0,2) [0,2)
Levels: [0,2) [2,4) [4,6]
```

The intervals are now defined how you want. Finally, you can add better labels to the categories by passing a character string vector to the `labels` argument, rather than using the interval levels that R applies by default:

```
R> lab <- c("Small","Medium","Large")
R> cut(x=Y,breaks=br,right=FALSE,include.lowest=TRUE,labels=lab)
[1] Small Large Small Medium Small Small Medium Large Small Small
Levels: Small Medium Large
```

The order of your supplied labels must match the order of the levels in the factor object.

Exercise 4.5

At the time of writing, the New Zealand Parliament consists of the political parties Labour, National, Greens, ACT, and Māori. There are several other parties in the political landscape that are not represented in parliament, labeled as Other. Suppose you asked 20 New Zealanders which party they identified most with and obtained the following data:

- There were 12 males and 8 females; the individuals numbered 1, 5–7, 12, and 14–16 were females.
 - The individuals numbered 1, 4, 12, 15, 16, and 19 identified with National; no one identified with Māori; the individuals numbered 6 and 9 identified with Greens; 11 and 20 identified with ACT; individual 10 identified with Other; and the rest identified with Labour.
- a. Use your knowledge of vectors (for example, subsetting and overwriting) to create two character vectors: `sex` with possible entries "M" (male) and "F" (female) and `party` with possible entries "National", "Labour", "Greens", "Maori", "ACT", and "Other". Make sure to place the entries in the correct positions as outlined earlier.
 - b. Create two different factors using `sex` and `party`; name them `sex.fac` and `party.fac`. You should explicitly specify the `levels` argument for the latter (because the sample doesn't represent all possible categories).
 - c. Identify the numeric representation of `party.fac`.

- d. Use subsetting to do the following:
 - i. Return the factor vector of chosen parties for only the male participants.
 - ii. Return the factor vector of genders for those who chose National.
- e. Another six people joined the survey, with the results `c("National", "Maori", "Maori", "Labour", "ACT", "Labour")` for the preferred party and `c("M", "M", "F", "F", "F", "M")` as their gender. Create new factors (named `party.fac2` and `sex.fac2`) that combine these results with the original factors from (b).
- f. As a brief aside, reconsider the original `party.fac` factor created in (b). Tabulate the factor: Are any categories associated with “zero counts”? If so, create another version named `party.fac3` that strips away any unpopulated categories from `party.fac`. Confirm via tabulation that `party.fac3` doesn’t contain zero-count levels.

Suppose you also asked all individuals to state how confident they were that National will win more seats in Parliament than Labour in the next election and to attach a subjective percentage to that confidence. You obtained the following results, corresponding to each of the 26 individuals now present in the sample: 75, 55, 20, 100, 10, 84, 50, 99, 33, 52, 35, 53, 25, 30, 100, 80, 56, 0, 64, 31, 10, 29, 40, 5, 18, 33.

- g. From these values, create a factor with levels of confidence as follows: Low for percentages [0,30]; Moderate for percentages (30,70]; and High for percentages (70,100].
- h. From the factor created in (g), use `party.fac2` created in (e) to extract the levels corresponding to those individuals who originally said they identified with Labour. Do this also for National. Is there anything to note about the levels of these sub-groups?

4.4 Dates and Times

Sometimes, your analysis requires dealing with dates and times. This could be for something as simple as annotating a plot, or something more involved where calculations depend on specific dates. For the former it usually suffices to use plain character strings, but in the case of the latter, you need more specialized objects.

Dates (and date-times) are complex, and we’ll look only at some simple features of base R here. As your needs become more sophisticated it’s often beneficial to consider additional R packages designed specifically for dealing with date-times, such as `lubridate` (Grolemund & Wickham 2011).

4.4.1 Creating and Formatting Dates

Let's start simple. You can write a date down in many different ways and store it as a character string. For example, the following objects `d1`, `d2`, and `d3` all represent the same date:

```
R> d1 <- "2025-02-19"
R> d2 <- "2/19/25"
R> d3 <- "19Feb2025"
```

As character strings, you can't perform any operations with these objects beyond using them as names or labels. To instruct R to interpret them as proper calendar dates, use the `as.Date` function and, in so doing, ensure R knows precisely how you've formatted the string so it correctly identifies the year, month, and day. The default is a "year-month-day" format; specifically:

YYYY-MM-DD or YYYY/MM/DD

In the strings, only `d1` is in one of these specific forms. We tell R to treat it as a proper date with the following call:

```
R> date1 <- as.Date(x=d1)
R> date1
[1] "2025-02-19"
```

At a glance, it seems nothing has changed; `date1` looks just like the character string `d1` from which it was created. But internally, the *class* of the object has changed, meaning you can ultimately do things with it that make sense in terms of calendar dates (you'll look at more technical details related to object classes in Section 6.2.2).

If the strings representing the dates are in a non-default format, you must use the optional `format` argument in the `as.Date` function. To this argument, supply another character string made up of *conversion specification* tags that tell `as.Date` how to interpret the date you've given it. There are a host of different conversion specifications; Table 4-5 summarizes a few of the most common and what they represent. Find a full list by inspecting the documentation file obtained by typing `?strptime` at the R prompt.

Table 4-5: Common Date-Specific Conversion Specifications for Date-Time Objects

Date marker	Role
%Y	Year as numeric YYYY
%y	Year as numeric YY
%m	Month as numeric MM
%B	Month as full text (as per <code>month.name</code>)
%b	Month as abbreviation (as per <code>month.abb</code>)
%d	Day as numeric DD

To see some of these conversion specifications in action, convert `d2` and `d3` to proper R dates as follows:

```
R> date2 <- as.Date(x=d2,format="%m/%d/%y")
R> date2
[1] "2025-02-19"
R> date3 <- as.Date(x=d3,format="%d%b%Y")
R> date3
[1] "2025-02-19"
```

In creating `date2`, you've told `as.Date` to expect `x` as a numeric month (`%m`), a forward slash `/`, the day (`%d`), another forward slash `/`, and a two-digit year (`%y`). For `date3`, you tell `as.Date` to expect the date as day (`%d`), abbreviated month (`%b`), and a four-digit year (`%Y`). Looking at how R prints the resulting objects, `date2` and `date3` are now entirely equivalent to `date1`: they all represent February 19, 2025.

Note that R stores, as built-in constants, full and abbreviated versions of the calendar months (in the locale of the current system) as the objects `month.name` and `month.abb`, respectively. These are immediately accessible at the R prompt, and you can use them as a reference for the `%B` and `%b` conversion specifications:

```
R> month.name
[1] "January" "February" "March" "April" "May" "June"
[7] "July" "August" "September" "October" "November" "December"
R> month.abb
[1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

You can store multiple dates in vectors. Suppose the following strings are the birthdates of five individuals:

```
R> birthstrings <- c("Jul-4-1985", "Mar-3-1954", "Jul-8-1955", "Oct-20-1988",
                    "Jul-26-2019")
```

You can convert them to dates all at once with the line

```
R> DOB <- as.Date(x=birthstrings,format="%b-%d-%Y")
R> DOB
[1] "1985-07-04" "1954-03-03" "1955-07-08" "1988-10-20" "2019-07-26"
```

and the usual rules regarding vector extraction/subsetting (cf. Section 2.3.3) apply:

```
R> DOB[3]
[1] "1955-07-08"
R> which(DOB>as.Date("1985-08-01"))
[1] 4 5
```

The first line provides the date at the third position in `DOB`, and the second line uses a relational operator (Section 4.1.2) in conjunction with the `which`

function (Section 4.1.5) to establish that only the fourth and fifth entries of `DOB` are dates that come after (are greater than) August 1, 1985.

You can interrogate any date object for the components that comprise it using the same conversion specifications in Table 4-5, returning the results as character strings. To do this, use `format` as a function and stipulate the specific contents of the date you want to an argument of the same name. For example, using the `DOB` date vector, the line

```
R> format(x=DOB,format="%d")
[1] "04" "03" "08" "20" "26"
```

picks out just the days of the five specific dates in `DOB`. You can find strings of the four-digit years like so:

```
R> format(x=DOB,format="%Y")
[1] "1985" "1954" "1955" "1988" "2019"
```

You're also given freedom to request more complicated combinations. For example, the following line returns both the abbreviated month and two-digit year of the requested date values:

```
R> format(x=DOB,format="%b %y")
[1] "Jul 85" "Mar 54" "Jul 55" "Oct 88" "Jul 19"
```

Other useful functions for interrogating dates include `weekdays` and `quarters`, which also return character strings of the corresponding weekday or yearly quarter of their corresponding argument:

```
R> weekdays(x=DOB)
[1] "Thursday" "Wednesday" "Friday"    "Thursday" "Friday"
R> quarters(x=DOB)
[1] "Q3" "Q1" "Q3" "Q4" "Q3"
```

4.4.2 Performing Simple Operations with Dates

Now that you've learned how to create formal objects for dates, let's explore how they differ from basic character strings, using the `d1` and `date1` objects created in Section 4.4.1. Recall from Section 4.2.1 that attempting, say, an arithmetic operation on a character string rightly results in an error:

```
R> d1
[1] "2025-02-19"
R> d1 + 1
Error in d1 + 1 : non-numeric argument to binary operator
```

But look what happens if you instead try to “add 1” to the formal date object of `d1`, namely `date1`:

```
R> date1
[1] "2025-02-19"
```

```
R> date1 + 1
[1] "2025-02-20"
```

Here, the operation returns a new date in which the day has been incremented by one.

The preceding example hints to the underlying numeric representation of a date object. Consider the result of the `as.numeric` function directly on `date1`:

```
R> as.numeric(date1)
[1] 20138
```

The resulting number expresses the given date in terms of the number of days since January 1, 1970 (known as the “Unix epoch”). This is clear if you look at the numeric form of this particular date:

```
R> as.numeric(as.Date(x="1970-01-01"))
[1] 0
```

Dates before the Unix epoch have a corresponding negative number; for example, the following line shows Christmas Day in 1969 as being seven days prior to this point:

```
R> as.numeric(as.Date(x="1969-12-25"))
[1] -7
```

The utility of the date format in this sense now starts to become apparent. If your desired tasks depend upon measuring time from some fixed point, then you needn’t explicitly concern yourself with the differing days of varying months or leap years; R handles all of that based on the dates of interest. For example, although `date1 + 1` is trivial, suppose you need to calculate the date exactly 139 days from February 19, 2025. You can find this as July 8 of the same year:

```
R> date1 + 139
[1] "2025-07-08"
```

In much the same fashion, it’s also easy to see how many days separate two dates:

```
R> term <- as.Date(x="1947-07-30")
R> ramb <- as.Date(x="1946-07-06")
R> term - ramb
Time difference of 389 days
```

Vectorized operations also hold true. Return to the five hypothetical individuals with birthdates stored in the `D08` object, created in Section 4.4.1.

Suppose you need to find out precisely how old each of them was, in days, on February 19, 2025:

```
R> date1 - DOB
Time differences in days
[1] 14475 25921 25429 13271 2035
```

You can also use `as.Date` to “reverse” a numeric (day) representation of a date to the corresponding calendar date, provided you supply an appropriate reference date (against which R will compare the number) to the `origin` argument (this can be a character string in a default date form or an existing date object). For example, the line

```
R> as.Date(x=481,origin="1970-01-01")
[1] "1971-04-27"
```

tells you that the 481st day after the Unix epoch is April 27, 1971; similarly, the line

```
R> as.Date(x=481,origin=date1)
[1] "2026-06-15"
```

informs you that June 15, 2026 is the date that occurs 481 days after February 19, 2025 (`date1`). Note that at the time of writing there is no default for `origin` when using `as.Date` in this way, so you must specify it each time.

4.4.3 *Incorporating Time*

The principles from the previous section carry through to full date-time definitions, where you can additionally specify hours, minutes, seconds, and even fractions of seconds. Unsurprisingly, this comes with additional complexity. Definitions of and calculations involving such objects must respect time zones, which in turn can raise even more issues, such as handling daylight savings. For these reasons we’ll look at a couple of simple examples here.

First, consider the following command, which, when you execute it at the R prompt, gives the present date and time as reported by your local machine (your result will be different to that displayed here!):

```
R> Sys.time()
[1] "2022-03-14 14:15:08 NZDT"
```

I wrote this sentence shortly after a quarter past two in the afternoon on New Zealand daylight saving time, March 14, 2022. The output hints at the standard form for full date-time specifications; following the date itself, the time is given as two-digit 24-hour, minute, and second components separated by colons, and there is clearly an allowance for time zone.

Given a character string provided in this way, the `strptime` function can create the corresponding date-time object. For example, you can create 34 minutes and 12 seconds past 7 PM on October 31, 2022 as follows:

```
R> dtime1 <- strptime(x="2022-10-31 19:34:12",format="%Y-%m-%d %H:%M:%S")
R> dtime1
[1] "2022-10-31 19:34:12 NZDT"
```

Here, note two things: First, you use `strptime` in the same way as `as.Date`, whereby the first argument is a string made up of the desired date-time components, and the second uses recognized conversion specifications to define the format (see Table 4-6).

Table 4-6: Common Time-Specific Conversion Specifications for Date-Time Objects

Time marker	Role
%H	Hours (in 2-digit, 24-hour form)
%M	Minutes (as 2-digit numeric)
%S	Seconds (as 2-digit numeric)

Second, the created object has been assigned a time zone; by default, R queries your system locale. In this case, it's correctly identified I'm in New Zealand, and the specified date falls during daylight saving time (NZDT).

You can dictate specific time zones using the optional `tz` argument in `strptime`. You can instruct R to store the same date-time as Greenwich Mean Time / UTC by supplying the string "GMT":

```
R> dtime2 <- strptime(x="2022-10-31 19:34:12",format="%Y-%m-%d %H:%M:%S",
                     tz="GMT")
R> dtime2
[1] "2022-10-31 19:34:12 GMT"
```

By typing `OlsonNames()` at your R prompt, you're provided the collection of time zone values able to be supplied to `tz` in a command like `strptime`. Query what R identifies your present time zone to be by typing `Sys.timezone()` at the prompt.

As best it can (there are various subtleties that can impart discrepancies between systems), R automatically identifies daylight savings statuses based on the provided date. For example, using the `OlsonNames()` entry for US Pacific time, note that the same date-time yields a PDT suffix, denoting Pacific daylight saving:

```
R> dtime3 <- strptime(x="2022-10-31 19:34:12",format="%Y-%m-%d %H:%M:%S",
                      tz="US/Pacific")

R> dtime3
[1] "2022-10-31 19:34:12 PDT"
```

However, if you set a date-time outside the usual daylight saving part of the year using the same time zone, like Christmas Day, R identifies the resulting object as Pacific Standard Time:

```
R> dtime4 <- strptime(x="2022-12-25 08:45:00",format="%Y-%m-%d %H:%M:%S",
                      tz="US/Pacific")

R> dtime4
[1] "2022-12-25 08:45:00 PST"
```

As always, these seemingly minor technicalities aren't just cosmetic; arguably the most important consequences arise when using the date-time objects in calculations. Taking the difference between `dtime1` and `dtime2` (which are defined to be exactly the same date-time, just in different time zones), for example, tells you that in New Zealand daylight saving time is 13 hours *ahead* of GMT:

```
R> dtime1 - dtime2
Time difference of -13 hours
```

Despite the order of differencing, the result is given as a negative here, implying that the New Zealand Daylight Time version, `dtime1`, needs 13 hours *added* to it in order for there to be zero difference between the two zones.

NOTE

Remember, daylight saving times are introduced, removed, or changed often at the whim of governments. As such, it's quite possible that the exact results of the preceding examples may well change when you run them in the future!

Finally, as earlier, note there is an underlying numeric representation of a full date-time object. In Section 4.4.2 this was provided in days since the Unix epoch (January 1, 1970) for date-only values. For full date-times, the default use of `as.numeric` function provides the converted value in terms of the number of *seconds* since January 1, 1970, at 00:00:00 GMT:

```
R> as.numeric(dtime1)
[1] 1667198052
```

Exercise 4.6

Aside from the usual Christmas and New Year periods, New Zealand presently has seven statutory public holidays. In the years 2024 and 2025, their observed dates were as follows:

Holiday	2024	2025
Waitangi Day	Feb. 6	Feb. 6
Good Friday	Mar. 29	Apr. 18
Easter Monday	Apr. 1	Apr. 21
ANZAC Day	Apr. 25	Apr. 25
King's Birthday	Jun. 3	Jun. 2
Matariki	Jun. 28	Jun. 20
Labour Day	Oct. 28	Oct. 27

- a. Use `as.Date` to create two date vectors: one for the 2024 holidays and one for the days in 2025.
- b. Use your two date vectors from (a) to complete the following tasks.
 - i. Find the weekdays corresponding to each of the holidays in both 2024 and 2025. By inspecting the output (or, more formally, using `which`), establish whether they're exactly the same for each holiday. Which holidays, if any, are on different days of the week between the two years?
 - ii. Repeat the previous task, but for the *quarters* of the year in which each holiday falls.
 - iii. Format the 2024 date vector to obtain just the months in abbreviated letter form, and the 2025 date vector to obtain the days in two-digit form.
 - iv. Exactly how many days separate Waitangi Day and King's Birthday in 2024?

- v. Exactly how many days separate Matariki in 2024 with the same holiday in 2025?
- vi. How many “sleeps” must an impatient child wait from Labour Day until Christmas Day (December 25) in 2025?

In Semester 1 of a given academic year, I am usually involved in lecturing STAT372, a course at the University of Otago on stochastic modeling, and the Semester 2 course STAT260, which deals with data visualization and R programming. In 2022, the last lecture of STAT372 commenced at 11 AM, Friday, May 27, and the first lecture of STAT260 commenced at 1 PM, Monday, July 11.

- c. Create two separate date-time objects representing the times (accurate to the second) and dates of the last lecture of STAT372 and the first lecture of STAT260, being sure to set the correct time zone to New Zealand time (simply identified as “NZ” in the output of `OlsonNames()`).
- d. Use your date-time objects from (c) to complete the following tasks:
 - i. Determine the time, in seconds, that elapsed from the Unix epoch to the first lecture of STAT260.
 - ii. Each lecture at the University of Otago is 50 minutes in duration. Noting that arithmetic operations on your date-time objects will be performed in *seconds*, use an appropriately scaled addition operation on your STAT372 object to create a new date-time object that represents the time and date of the *end* of the last lecture of STAT372. Inspect the resulting object to ensure it shows the correct ending time.
 - iii. Using your object from (ii), determine the duration of relative calm I may enjoy between the end of lecturing STAT372 and the beginning of STAT260.

Important Code in This Chapter

Function/operator	Brief description	First occurrence
TRUE, FALSE	Reserved logical values	Section 4.1.1, p. 60
T, F	Unreserved versions of logicals	Section 4.1.1, p. 60
==, !=, >, <, >=, <=	Relational operators	Section 4.1.2, p. 61
any	Check whether any entries are TRUE	Section 4.1.2, p. 63
all	Check whether all entries are TRUE	Section 4.1.2, p. 63
&&, &, , , !	Logical operators	Section 4.1.3, p. 65
which	Determine indexes of TRUEs	Section 4.1.5, p. 69
" "	Designate a character string	Section 4.2.1, p. 73
nchar	Get number of characters in a string	Section 4.2.1, p. 73
cat	Concatenate strings (no return)	Section 4.2.2, p. 74
paste	Paste strings (return a string)	Section 4.2.2, p. 74
paste0	As per paste, with sep="" by default	Section 4.2.2, p. 76
\	Invoke escape sequence inside string	Section 4.2.3, p. 77
grep	String matching	Section 4.2.4, p. 78
substr	Subset a string	Section 4.2.4, p. 79
sub, gsub	String matching and replacement	Section 4.2.4, p. 79
factor	Create a factor vector	Section 4.3.1, p. 82
levels	Get levels of a factor	Section 4.3.1, p. 82
table	Tabulate/count category membership	Section 4.3.2, p. 84
relevel	Set first (reference) level of factor	Section 4.3.2, p. 85
droplevels	Strip away unused levels of factor	Section 4.3.3, p. 86
cut	Create factor from continuum	Section 4.3.7, p. 90
as.Date	Create date object	Section 4.4.1, p. 93
%Y, %y, %m, %B, %b, %d	Date conversion specifications	Section 4.4.1, p. 93
month.name, month.abb	Built-in month name objects	Section 4.4.1, p. 94
format	Extract components of a date	Section 4.4.1, p. 95
weekdays	Identify weekday of a date	Section 4.4.1, p. 95
quarters	Identify quarter of a date	Section 4.4.1, p. 95
as.numeric	Numeric representation of factor	Section 4.4.2, p. 96
Sys.time	Report your local date-time	Section 4.4.3, p. 97
strptime	Create date-time object	Section 4.4.3, p. 98
%H, %M, %S	Time conversion specifications	Section 4.4.3, p. 98
OlsonNames	Olson time zone names	Section 4.4.3, p. 98